По умолчанию весь код, написанный пользователем в Unity использует всего 1 поток процессора. Учитывая, что у процессора потоков обычно больше, очевидно, что, если заставить остальные потоки тоже просчитывать какую-то логику параллельно с основным, производительность вырастет.

Для начала нужно определиться с тем, какие системы вообще можно и нужно «распараллелить». Параллельность нужна тогда, когда происходит много одинаковых операций над разными данными, и эти операции не ждут друг друга.

Например, система движения – для 1000 Entity необходимо как-то посчитать и поменять положение. Пусть всего у процессора 10 рабочих потоков, тогда на каждый пойдёт обрабатываться логика 100 Entity вместо того, чтобы последовательно обрабатываться на одном главном потоке.

А вот система спавна – отсчитывает время по таймеру и раз в X секунд создаёт нового Entity. В ней абсолютно нечего вычислять параллельно. Весь процесс – однородный и последовательный, а значит – можно обойтись и одним потоком, да и даже при желании не выйдет сделать это через параллельные процессы. Параллелить просто нечего.

Для первого примера реализации параллельной системы возьмём элементарную систему регенерации. Без параллельных вычислений она выглядит вот так:

```
[UpdateInGroup(typeof(RegenerationSystemGroup))]
Ссылок: 1
partial struct RegenerationSystem : ISystem
{
    [BurstCompile]
    Ссылок: 0
    public void OnUpdate(ref SystemState state)
    {
        foreach(var (regeneration, health) in SystemAPI.Query<RefRO<Regeneration>, RefRW<Health>>())
        {
            if(health.ValueRO.CurrentHealth < health.ValueRO.MaxHealth)
            {
                health.ValueRW.CurrentHealth = math.min(health.ValueRO.MaxHealth, health.ValueRO.CurrentHealth
                    +regeneration.ValueRO.HealthRegeneratedPerUpdate);
            }
        }
    }
}
```

Чтобы сделать систему параллельной необходимо создать структуру Job (далее джоба), которая будет описывать процесс, происходящий на каждом рабочем потоке. Интерфейс IJobEntity обозначает, что это будет параллельный процесс, выполняющийся для каждого подходящего Entity, аналогично однопоточной (последовательной) системе в цикле foreach.

```
[BurstCompile]
Ссылок: 15
public partial struct RegenerationJob : IJobEntity
{

}
```

Сама логика, которая будет происходить в «джобе» (в каждом отдльном потоке для каждого Entity), будет находиться внутри метода Execute.

```
[BurstCompile]
Ссылок: 15
public partial struct RegenerationJob : IJobEntity
{
    [BurstCompile]
    Ссылок: 4
    public void Execute()
    {

    }
}
```

Ранее, при работе с SystemAPI.Query, чтобы пройтись по Entity, имеющим определённые компоненты, было необходимо использовать вспомогательные структуры RefRO и RefRW. В случае с джобами и методом Execute, необходимые компоненты можно просто передать параметрами в метод с ключевыми словами ref и in для чтения и записи или только чтения соответственно. Вместо метода WithEntityAccess, достаточно передать Entity параметром метода Execute. В сравнении с последовательным способом выполнения, запись даже облегчилась.

```
[BurstCompile]
Ссылок: 15
public partial struct RegenerationJob : IJobEntity
{
    [BurstCompile]
    Ссылок: 4
    public void Execute(in Regeneration regeneration, ref Health health)
    {
        if (health.CurrentHealth < health.MaxHealth)
        {
            health.CurrentHealth = math.min(health.MaxHealth, health.CurrentHealth + regeneration.HealthRegeneratedPerUpdate);
        }
    }
}
```

Но на данный момент эта джоба – просто созданная структура, которая не будет работать сама по себе. Задачу запустить параллельный процесс возьмёт на себя сама система, в которой раньше содержалась логика. Теперь задача этой системы – не выполнить логику, а создать и запланировать джобу.

```
[UpdateInGroup(typeof(RegenerationSystemGroup))]
Ссылок: 1
partial struct RegenerationSystem : ISystem
{
    [BurstCompile]
    Ссылок: 0
    public void OnUpdate(ref SystemState state)
    {
        var job = new RegenerationJob()
        {
        };
        state.Dependency = job.ScheduleParallel(state.Dependency);
    }
}

[BurstCompile]
Ссылок: 15
public partial struct RegenerationJob : IJobEntity
{
    [BurstCompile]
    Ссылок: 4
    public void Execute(in Regeneration regeneration, ref Health health)
    {
        if (health.CurrentHealth < health.MaxHealth)
        {
            health.CurrentHealth = math.min(health.MaxHealth, health.CurrentHealth
                + regeneration.HealthRegeneratedPerUpdate);
        }
    }
}
```

Что значит строка:

«state.Dependency = job.ScheduleParallel(state.Dependency);»?

В ней происходит планирование джоба. Дело в том, что тело метода Execute из структуры RegenerationJob не выполнится мгновенно, как только выполнение дойдёт до этой строки. Джоб лишь добавится в очередь планировщика работ. Очередь формируется из точно таких же джобов, запускаемых из других систем.

state.Dependency – это поле каждой системы типа JobHandle, которое хранит информацию о зависимостях этой системы: какие джобы должны завершиться перед продолжением работы.

job.ScheduleParallel(state.Dependency) ставит RegenerationJob в очередь. Этот джоб не начнёт выполняться, пока не завершится джоб, на который указывает state.Dependency.

Присваивание state.Dependency = job.ScheduleParallel(state.Dependency) передаёт следующую зависимость дальше: следующая система увидит, что RegenerationJob тоже встал в очередь, и будет дожидаться его завершения.

job.ScheduleParallel(state.Dependency) – по сути ставит RegenerationJob в очередь. Эта джоба не начнёт выполняться, пока не завершится процесс, наблюдаемый state.Dependency. а state.Dependency = job.ScheduleParallel(state.Dependency) позволяет следующим системам увидеть, что RegenerationJob тоже встал в очередь, а значит, следующая система будет дожидаться уже его завершения.

В связи с этим следует главный вывод – то, что написано после job.ScheduleParallel в OnUpdate, совершенно не обязательно выполнится после тела метода Execute. Даже наоборот, почти гарантированно оно выполнится раньше.

Стоит отметить также, что вместе с циклом foreach в последовательных системах в комплекте шло немалое количество удобных методов для фильтрации необходимых Entity (например WithDisabled, WithNone итд). Чтобы воспользоваться подобной более тонкой настройкой EntityQuery, возможно создать этот EntityQuery отдельно в системе с помощью QueryBuilder, а затем передать созданный EntityQuery как параметр метода ScheduleParallel вместе со state.Dependency. Альтернативой для метода WithChangeFilter является называющийся точно также атрибут, который пишется над джобой.

```
[UpdateAfter(typeof(MovementSystem))]
Ссылок: 1
partial struct HealthSystem : ISystem
{
    EntityQuery healthSystemQuery;

    [BurstCompile]
    Ссылок: 0
    public void OnCreate(ref SystemState state)
    {
        healthSystemQuery = SystemAPI.QueryBuilder().WithAll<Health>().WithDisabled<Dead>().Build();
    }
    [BurstCompile]
    Ссылок: 0
    public void OnUpdate(ref SystemState state)
    {

        var job = new HealthJob()
        {
        };
        state.Dependency = job.ScheduleParallel(healthSystemQuery ,state.Dependency);


    }
}
```

Процесс работы с EntityCommandBuffer также немного отличается в связи со спецификой параллельных вычислений. Прежде всего – джобы используют EntityCommandBuffer.ParallelWriter вместо обычного EntityCommandBuffer. ParallelWriter-версия требует в своих методах специальный индекс, который позволяет впоследствии командам, вызванным параллельно с разных потоков, определить чёткий порядок выполнения при EntityCommandBuffer.Playback(). Этот индекс также записывается как параметр метода Execute со специальным атрибутом [ChunkIndexInQuery].

```
[UpdateAfter(typeof(MovementSystem))]
Ссылок: 1
partial struct HealthSystem : ISystem
{
    EntityQuery healthSystemQuery;

    [BurstCompile]
    Ссылок: 0
    public void OnCreate(ref SystemState state)
    {
        healthSystemQuery = SystemAPI.QueryBuilder().WithAll<Health>().WithDisabled<Dead>().Build();
    }
    [BurstCompile]
    Ссылок: 0
    public void OnUpdate(ref SystemState state)
    {
        EntityCommandBuffer ecb = new EntityCommandBuffer(Allocator.TempJob);

        var job = new HealthJob()
        {
            ecb = ecb.AsParallelWriter()
        };
        state.Dependency = job.ScheduleParallel(healthSystemQuery ,state.Dependency);

        state.Dependency.Complete();
        ecb.Playback(state.EntityManager);
        ecb.Dispose();

    }
}

[WithChangeFilter(typeof(Health))]
Ссылок: 15
public partial struct HealthJob : IJobEntity
{
    public EntityCommandBuffer.ParallelWriter ecb;
    Ссылок: 4
    void Execute([ChunkIndexInQuery] int chunkIndexInQuery,Entity entity, in Health health)
    {
        if (health.CurrentHealth <= 0)
        {
            ecb.SetComponentEnabled<Dead>(chunkIndexInQuery, entity, true);
        }
    }
}
```

Важнейшим изменением с ECB в контексте параллельных процессов является тот факт, что на него накладывается одно огромное ограничение: Playback нельзя воспроизвести до тех пор, пока Unity не будет уверено, что

джоба, использующая этот ECB, закончила своё выполнение. Учитывая, что, как было сказано ранее, код, воспроизведённый после ScheduleParallel не даёт никаких гарантий завершённости самой джобы, просто написать Playback там не выйдет. Придётся принудительно дожидаться завершения процесса, использующего этот ECB. Делается это с помощью state.Dependency.Complete(). Безусловно, это убивает значительную часть выигрыша в производительности, который получается путём асинхронного планирования джобов, поэтому использовать ECB следует по возможности по минимуму, хотя, конечно, совсем без него проект сделать будет невозможно.

```
[UpdateAfter(typeof(MovementSystem))]
Ссылок: 1
partial struct HealthSystem : ISystem
{
    EntityQuery healthSystemQuery;

    [BurstCompile]
    Ссылок: 0
    public void OnCreate(ref SystemState state)
    {
        healthSystemQuery = SystemAPI.QueryBuilder().WithAll<Health>().WithDisabled<Dead>().Build();
    }
    [BurstCompile]
    Ссылок: 0
    public void OnUpdate(ref SystemState state)
    {
        EntityCommandBuffer ecb = new EntityCommandBuffer(Allocator.TempJob);

        var job = new HealthJob()
        {
            ecb = ecb.AsParallelWriter()
        };
        state.Dependency = job.ScheduleParallel(healthSystemQuery ,state.Dependency);

        state.Dependency.Complete();
        ecb.Playback(state.EntityManager);
        ecb.Dispose();

    }
}
```

ComponentLookup, использовавшийся и ранее как альтернатива метода GetComponent, также приобретает ряд ограничений в контексте параллельных джобов. Конкретно – можно использовать ComponentLookup только для чтения. Объясняется это тем, что ComponentLookup используется, чтобы получить компонент какого-то «другого» Entity, не того, которого сейчас обрабатывает Execute. А это значит, что если разрешить ComponentLookup писать в компонент этого другого Entity, можно получить ошибку, называемую гонкой данных – если в один и тот же компонент одного и того же Entity

одновременно решат написать 2 или больше разных потока процессора. Результат будет непредсказуем. Чтобы «успокоить» Unity в этом отношении, используется атрибут Unity.Collections.ReadOnly.

```
partial struct SelectTowerSystem : ISystem
{
    [BurstCompile]
    Ссылок: 0
    public void OnUpdate(ref SystemState state)
    {
        if(!SystemAPI.TryGetSingletonEntity<Tower>(out Entity towerEntity))
        {
            return;
        }

        ComponentLookup<LocalTransform> transformLookup = SystemAPI.GetComponentLookup<LocalTransform>(true);

        var job = new SelectTowerJob()
        {
            Tower = towerEntity,
            TransformLookup = transformLookup
        };

        state.Dependency = job.ScheduleParallel(state.Dependency);
    }
}
Ссылок: 15
public partial struct SelectTowerJob : IJobEntity
{
    [ReadOnly] public ComponentLookup<LocalTransform> TransformLookup;
    public Entity Tower;
    Ссылок: 4
    void Execute(ref Movement movement)
    {
        movement.TargetPosition = TransformLookup[Tower].Position;
    }
}
```

Но как быть, если система, обрабатывающая одного Entity, должна менять другого Entity? Простые структурные изменения типа добавления или удаления компонента или удаления самого Entity – с этими задачами легко справится EntityCommandBuffer. Но есть ситуации, в которых придётся танцевать с бубном. Например, башни атакуют противников. Система работает для башни, но меняться из-за атаки должно здоровье противника. При этом, если башен может быть несколько, то в одном кадре по одному противнику может прилететь урон, например, сразу от 3 башен. Так как каждая из башен не знает о том, что остальные башни тоже атаковали противника, каждая башня просто запишет с помощью ecb.SetComponent новое значение здоровья противника «health = (health – 10)». И получается, что башни должны были за эти 3 попадания в 1 кадре отнять 30 очков здоровья у противника, а отнимут только 10.

Решением такой ситуации является создание буфера урона – списка всех атак, произошедших в этом кадре, которые запишутся в буфер параллельно, но выполнятся последовательно.

Для создания такого буфера в данном примере используется NativeList. Это Native-контейнер – коллекция, которая хранит данные вне управляемой памяти и не зависит от сборщика мусора. Благодаря этому такие контейнеры можно безопасно использовать в DOTS и как следствие, джобах. Плата за это – ручное управление памятью с помощью Allocator и своевременного Dispose, как и в случае с ECB. Чаще всего такие Native-контейнеры (NativeArray, NativeList, NativeReference, NativeParallelMultiHashMap итд) создаются и уничтожаются непосредственно в системах или джобах и обычно используются для задач непосредственно в процессе выполнения логики. По умолчанию их хранение в компонентах не является чем-то распространённым, и злоупотреблять этим не нужно, поскольку Native-контейнеры по сути своей, хотя и являются структурами, ведут себя очень похоже на ссылочные типы, ведь внутри них хранится указатель на память. Единственный сценарий, когда хранить их в компонентах приемлемо – как в данном примере – с синглтон компонентами.

```
class TowerDamageHolderAuthoring : MonoBehaviour
{
    Ссылок: 0
    class TowerDamageHolderBaker : Baker<TowerDamageHolderAuthoring>
    {
        Ссылок: 0
        public override void Bake(TowerDamageHolderAuthoring authoring)
        {
            Entity entity = GetEntity(TransformUsageFlags.None);

            AddComponent(entity, new TowerDamageHolder()
            {
                damages = new NativeList<TowerDamage>(1024, Allocator.Persistent)
            });
        }
    }
}

[ChunkSerializable]
Ссылок: 9
public struct TowerDamageHolder : IComponentData
{
    public NativeList<TowerDamage> damages;
}

Ссылок: 10
public struct TowerDamage
{
    public Entity DamageTarget;
    public int Damage;
}
```

Теперь системе, которая параллельно записывает взаимодействия башни и противника, достаточно добавить их в NativeList.

Большой и страшный атрибут [NativeDisableContainerSafetyRestriction] отключает систему безопасности Unity в отношении нужного Native-контейнера. Нужно это потому, что в нашем примере NativeList используется двумя разными джобами: первая джоба записывает в него данные, вторая – читает. Unity видит это и предупреждает: «А вдруг первая джоба ещё не закончила запись, а вторая уже начала чтение или того хуже, тоже запись? Это может привести к гонке данных!» Однако в данном случае точно известно, что цепочка зависимостей (state.Dependency) гарантирует: вторая джоба не запустится, пока первая не завершится. Поэтому можно «успокоить» систему безопасности этим атрибутом.

Важно понимать: использование этого атрибута – это передача ответственности за безопасность памяти с Unity на программиста. Если вы ошибётесь с зависимостями или допустите параллельный доступ, получите краш или неопределённое поведение. Используйте этот атрибут только тогда, когда полностью понимаете, что делаете и почему это безопасно!

```
partial struct TowerAttackQuerySystem : ISystem
{
    [BurstCompile]
    Ссылок: 0
    public void OnUpdate(ref SystemState state)
    {
        NativeList<TowerDamage> damages = SystemAPI.GetSingleton<TowerDamageHolder>().damages;
        var job = new TowerAttackQueryJob()
        {
            Damages = damages.AsParallelWriter()
        };
        state.Dependency = job.ScheduleParallel(state.Dependency);
    }
    [BurstCompile]
    Ссылок: 0
    public void OnDestroy(ref SystemState state)
    {
        SystemAPI.GetSingleton<TowerDamageHolder>().damages.Dispose();
    }
}
[BurstCompile]
Ссылок: 15
public partial struct TowerAttackQueryJob : IJobEntity
{
    [NativeDisableContainerSafetyRestriction] public NativeList<TowerDamage>.ParallelWriter Damages;
    [BurstCompile]
    Ссылок: 4
    public void Execute(in Tower towerData)
    {
        Damages.AddNoResize(new TowerDamage()
        {
            DamageTarget = towerData.CurrentTarget,
            Damage = 1
        });
    }
}
```

Теперь, чтобы последовательно, а не параллельно обработать полученные данные об уроне, необходимо воспользоваться не параллельным

джобом. И тут поможет IJob. В отличие от IJobEntity, IJob выполняется не на главном, но всё же только на 1 потоке, а следовательно, не может создать гонки данных. И ComponentLookup, соответственно, можно использовать и для чтения, и для записи. Важное отличие: IJobEntity тесно связан с ECS и автоматически перебирает Entity, подходящие под запрос из параметров метода Execute. IJob же – это универсальный последовательный джоб, который ничего не знает ни про какие Entity. Он просто выполняет метод Execute один раз. Если в нём нужны данные из компонентов – их нужно получать через ComponentLookup или передавать в джоб как поля.

```
[UpdateAfter(typeof(TowerAttackQuerySystem))]
Ссылок: 1
partial struct TowerAttackHandlerSystem : ISystem
{
    [BurstCompile]
    Ссылок: 0
    public void OnUpdate(ref SystemState state)
    {
        NativeList<TowerDamage> damages = SystemAPI.GetSingleton<TowerDamageHolder>().damages;
        ComponentLookup<Health> healthLookup = SystemAPI.GetComponentLookup<Health>(false);

        var job = new TowerAttackHandlerJob()
        {
            Damages = damages,
            HealthLookup = healthLookup
        };
        state.Dependency = job.Schedule(state.Dependency);
    }
}

[BurstCompile]
Ссылок: 2
public struct TowerAttackHandlerJob : IJob
{
    public NativeList<TowerDamage> Damages;
    public ComponentLookup<Health> HealthLookup;

    [BurstCompile]
    Ссылок: 0
    public void Execute()
    {
        foreach (TowerDamage damage in Damages)
        {
            if(HealthLookup.TryGetRefRW(damage.DamageTarget, out RefRW<Health> health))
            {
                health.ValueRW.CurrentHealth = math.max(0, health.ValueRO.CurrentHealth - damage.Damage);
            }
        }
        Damages.Clear();
    }
}
```